

BehaviorTree.CPP v3 — Kapsamlı Node Eğitim Rehberi

Teknofest 2026 Robotaksi Projesi

İÇİNDEKİLER

1. Behavior Tree Nedir?
2. Node Türleri (Action, Condition, Control, Decorator)
3. BT.CPP v3 Mimarisi — Sınıf Hiyerarşisi
4. Port Sistemi (Input, Output, Bidirectional)
5. Blackboard Nedir, Nasıl Çalışır?
6. SyncActionNode — En Basit Action
7. StatefulActionNode — Asenkron/Uzun Süreli Action
8. ConditionNode — Koşul Kontrolü
9. XML Syntax ve TreeNodesModel
10. Node Kayıt (Register) Süreci
11. Gerçek Dünya Örneği: Robotaksi Node'ları
12. Sık Yapılan Hatalar ve Çözümleri

1. BEHAVIOR TREE NEDİR?

Behavior Tree (BT), robotik ve oyun AI'ında kullanılan bir **karar ağacı** yapısıdır. Her "tick" (kalp atışı) döngüsünde ağaç kökten yapraklara doğru yürütülür.

Her node 3 durumdan birini döner:

- **SUCCESS**: Görev başarılı
- **FAILURE**: Görev başarısız
- **RUNNING**: Görev devam ediyor (henüz bitmedi)

Neden FSM (State Machine) Değil de BT?

Özellik	FSM	BT
Modülerlik	Zayıf (geçişler karışır)	Güçlü (alt ağaç taşınır)
Ölçeklenebilirlik	N^2 geçiş	Doğrusal büyüme
Reaktivite	Manuel	ReactiveSequence ile otomatik
Debug	Zor	Groot ile görsel

2. NODE TÜRLERİ

2.1 Control Node'ları (Dahili — Kendin Yazmıyorsun)

Sequence: Tüm çocuklar SUCCESS → SUCCESS (AND mantığı)
Bir çocuk FAILURE → hemen FAILURE

Fallback: Bir çocuk SUCCESS → hemen SUCCESS (OR mantığı)
Tüm çocuklar FAILURE → FAILURE

ReactiveSequence: Her tick'te BAŞTAN kontrol eder
(güvenlik refleksleri için ideal)

XML Örneği:

```
<Sequence name="Gorev">
  <KosulKontrol />    <!-- FAILURE dönerse Sequence durur -->
  <Aksiyonu_Yap />    <!-- Sadece koşul SUCCESS ise çalışır -->
</Sequence>
```

2.2 Decorator Node'ları (Dahili)

Inverter: SUCCESS ↔ FAILURE (NOT mantığı)
ForceSuccess: Her sonucu SUCCESS yapar
KeepRunningUntilFailure: SUCCESS → RUNNING, FAILURE → FAILURE
Repeat: N kez tekrarla

2.3 Action Node (SEN YAZIYORSUN — Yaprak)

Bir **iş yapan** node. Örnek: "Aracı durdur", "Far aç", "Hız ayarla"

2.4 Condition Node (SEN YAZIYORSUN — Yaprak)

Bir **koşul kontrol eden** node. Hiçbir yan etkisi OLMAMALI. Örnek: "Yaya var mı?", "Segment tipi nedir?", "Tur $\geq$ 2 mi?"

ÖNEMLİ: Condition ASLA RUNNING dönemez! Sadece SUCCESS veya FAILURE.

3. BT.CPP v3 SINIF HİYERARŞİSİ

BT::TreeNode ← Her şeyin atası
├─ BT::ControlNode ← Sequence, Fallback...
├─ BT::DecoratorNode ← Inverter, ForceSuccess...
├─ BT::ConditionNode ← SEN YAZIYORSUN (tick → S/F)
└─ BT::ActionNodeBase
 ├─ BT::SyncActionNode ← SEN YAZIYORSUN (tek tick)
 └─ BT::StatefulActionNode ← SEN YAZIYORSUN (çok tick)

SyncActionNode vs StatefulActionNode

Özellik	SyncActionNode	StatefulActionNode
Override	tick()	onStart(), onRunning(), onHalted()
RUNNING dönebilir mi?	HAYIR!	EVET
Kullanım	Anlık işler	Uzun süreli işler

Örnek	SetMaxSpeed, StopVehicle	FollowLane, Dwell, WaitForClear
-------	--------------------------	---------------------------------

4. PORT SİSTEMİ

Port'lar node'ların Blackboard ile veri alışverişi yapmasını sağlar.

4.1 Port Türleri

```
// Sadece okuma – XML'den veya Blackboard'dan değer alır
BT::InputPort<int>("tour", "Tur numarası")

// Sadece yazma – Blackboard'a değer yazar
BT::OutputPort<std::string>("route", "Hesaplanan rota")

// Hem okuma hem yazma
BT::BidirectionalPort<int>("seg_index")
```

4.2 XML'de Port Kullanımı

```
<!-- Sabit değer (literal) -->
<SetMaxSpeed speed="1.50" />

<!-- Blackboard referansı (süslü parantez) -->
<SetMaxSpeed speed="{cruise_speed}" />

<!-- Blackboard'a yaz -->
<SetBlackboard output_key="cruise_speed" value="1.50" />
```

Kural: {değişken_adı} → Blackboard'dan oku/yaz değer (parantezsiz) → Sabit literal değer

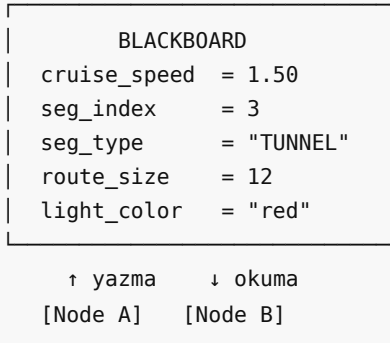
4.3 C++ Tarafında Port Okuma/Yazma

```
// InputPort okuma
double speed = 0.0;
auto result = getInput("speed", speed);
if (!result) {
    // Port okunamadı – hata yönetimi
    throw BT::RuntimeError("speed portu okunamadı: ", result.error());
}

// OutputPort yazma
setOutput("route", std::string("[{...}]"));
setOutput("seg_index", 0);
```

5. BLACKBOARD NEDİR?

Blackboard, ağaçtaki TÜM node'ların paylaştığı bir **anahtar-değer deposudur**.



- SetBlackboard XML node'u ile başlangıç değerleri atanır
- OutputPort ile node'lar değer yazar
- InputPort ile node'lar değer okur
- {port_name} syntax'ı Blackboard referansıdır

6. SyncActionNode — EN BASİT ACTION

Şablon (Boilerplate)

```
// — header (.hpp) —
class BenimAction : public BT::SyncActionNode {
public:
    // Kurucu — HER ZAMAN bu imzayı kullan
    BenimAction(const std::string& name, const BT::NodeConfiguration& config)
        : BT::SyncActionNode(name, config) {}

    // Port tanımları — XML ile birebir eşleşmeli
    static BT::PortsList providedPorts() {
        return {
            BT::InputPort<double>("hiz", "Hedef hız m/s"),
            BT::OutputPort<std::string>("sonuc", "İşlem sonucu")
        };
    }

    // İş yapan fonksiyon — BİR KERE çağrılır
    BT::NodeStatus tick() override;
};
```

```
// — source (.cpp) —
BT::NodeStatus BenimAction::tick()
{
    // 1. Input'ları oku
    double hiz = 0.0;
    getInput("hiz", hiz);

    // 2. İş yap
    RCLCPP_INFO(rclcpp::get_logger("bt"), "Hız ayarlandı: %.2f", hiz);
}
```

```
// 3. Output'ları yaz (varsa)
setOutput("sonuc", std::string("tamam"));

// 4. Sonuç dön – SUCCESS veya FAILURE
return BT::NodeStatus::SUCCESS;
}
```

Gerçek Örnek: SetMaxSpeed

```
// Bu node "anlık" bir iştir – ROS topic'e hız yayınla, bitti.
BT::NodeStatus SetMaxSpeed::tick()
{
    double speed = 0.0;
    getInput("speed", speed);

    // ROS2 ile hız parametresini güncelle
    // (gerçek implementasyonda dynamic_reconfigure veya topic)
    RCLCPP_INFO(logger(), "SetMaxSpeed: %.2f m/s", speed);

    return BT::NodeStatus::SUCCESS;
}
```

Gerçek Örnek: AdvanceSegment

```
// seg_index'i 1 artır – BidirectionalPort kullanır
BT::NodeStatus AdvanceSegment::tick()
{
    int seg_index = 0;
    getInput("seg_index", seg_index); // Oku
    seg_index++;                       // Değiştir
    setOutput("seg_index", seg_index); // Geri yaz
    RCLCPP_INFO(logger(), "AdvanceSegment: yeni index=%d", seg_index);
    return BT::NodeStatus::SUCCESS;
}
```

Gerçek Örnek: ClearHandledFlags

```
// Blackboard bayraklarını temizle – port'suz, yan etki: blackboard
BT::NodeStatus ClearHandledFlags::tick()
{
    // config().blackboard ile doğrudan Blackboard'a erişim
    auto bb = config().blackboard;
    bb->set("handled_stop_sign", false);
    bb->set("handled_traffic_light", false);
    bb->set("handled_road_sign", false);
    RCLCPP_DEBUG(logger(), "ClearHandledFlags: bayraklar sıfırlandı");
    return BT::NodeStatus::SUCCESS;
}
```

7. StatefulActionNode — UZUN SÜRELİ ACTION

RUNNING dönmesi gereken işler için kullanılır. 3 fonksiyon override edilir:

```
onStart() → İlk tick'te çağrılır (başlatma)
           SUCCESS/FAILURE/RUNNING dönebilir

onRunning() → Sonraki her tick'te çağrılır (devam kontrolü)
             SUCCESS → iş bitti
             FAILURE → hata oluştu
             RUNNING → devam ediyor

onHalted() → Ağaç durdurulursa çağrılır (temizlik)
            Dönüş değeri yok
```

Yaşam Döngüsü Diyagramı

```
tick #1 → onStart() → RUNNING
tick #2 → onRunning() → RUNNING
tick #3 → onRunning() → RUNNING
...
tick #N → onRunning() → SUCCESS ← İş bitti!

(eğer ara da halted olursa)
tick #K → onHalted() ← Temizlik yap
```

Gerçek Örnek: Dwell (Bekle)

```
// — onStart: zamanlayıcıyı başlat —
BT::NodeStatus Dwell::onStart()
{
    double min_sec = 15.0, max_sec = 20.0;
    getInput("min_sec", min_sec);
    getInput("max_sec", max_sec);

    // min ile max arasında rastgele veya ortalama süre
    wait_duration_sec_ = (min_sec + max_sec) / 2.0;
    start_time_ = std::chrono::steady_clock::now();

    RCLCPP_INFO(logger(), "Dwell: %.0f sn bekleniyor", wait_duration_sec_);
    return BT::NodeStatus::RUNNING; // ← Henüz bitmedi
}

// — onRunning: süre doldu mu kontrol et —
BT::NodeStatus Dwell::onRunning()
{
    auto elapsed = std::chrono::steady_clock::now() - start_time_;
    double sec = std::chrono::duration<double>(elapsed).count();

    if (sec >= wait_duration_sec_) {
```

```

        RCLCPP_INFO(logger(), "Dwell: bekleme tamamlandı");
        return BT::NodeStatus::SUCCESS; // ← İş bitti
    }
    return BT::NodeStatus::RUNNING;    // ← Devam et
}

// — onHalted: erken durdurulursa —
void Dwell::onHalted()
{
    RCLCPP_WARN(logger(), "Dwell: HALTED (bekleme kesildi)");
    // Gerekli temizlik (timer iptal, action cancel vs.)
}

```

Gerçek Örnek: WaitForGoSignal

```

BT::NodeStatus WaitForGoSignal::onStart()
{
    RCLCPP_INFO(logger(), "WaitForGoSignal: UMS-2 Go sinyali bekleniyor...");
    // ROS subscriber burada başlatılabilir
    return BT::NodeStatus::RUNNING;
}

BT::NodeStatus WaitForGoSignal::onRunning()
{
    // TODO: ROS topic'ten Go sinyali geldi mi kontrol et
    // if (go_signal_received_) return BT::NodeStatus::SUCCESS;
    // return BT::NodeStatus::RUNNING;

    // Stub: hemen başlat
    return BT::NodeStatus::SUCCESS;
}

void WaitForGoSignal::onHalted()
{
    RCLCPP_WARN(logger(), "WaitForGoSignal: HALTED");
}

```

8. ConditionNode — KOŞUL KONTROLÜ

Kurallar

1. ASLA RUNNING dönemez — sadece SUCCESS veya FAILURE
2. ASLA yan etkisi olmamalı (topic publish etmemeli, state değiştirmemeli)
3. Hızlı olmalı (her tick'te çağrılabilir)

Şablon

```

class BenimKosul : public BT::ConditionNode {
public:
    BenimKosul(const std::string& name, const BT::NodeConfiguration& config)

```

```

        : BT::ConditionNode(name, config) {}

static BT::PortsList providedPorts() {
    return { BT::InputPort<int>("deger") };
}

BT::NodeStatus tick() override;
};

BT::NodeStatus BenimKosul::tick()
{
    int deger = 0;
    getInput("deger", deger);
    // Koşul kontrolü – saf mantık, yan etki YOK
    return (deger > 10) ? BT::NodeStatus::SUCCESS
        : BT::NodeStatus::FAILURE;
}

```

Gerçek Örnek: HasMoreSegments

```

BT::NodeStatus HasMoreSegments::tick()
{
    int seg_index = 0, route_size = 0;
    getInput("seg_index", seg_index);
    getInput("route_size", route_size);

    // Saf karşılaştırma – yan etki yok
    return (seg_index < route_size) ? BT::NodeStatus::SUCCESS
        : BT::NodeStatus::FAILURE;
}

```

Gerçek Örnek: IsSegmentType

```

BT::NodeStatus IsSegmentType::tick()
{
    std::string seg_type, expected;
    getInput("seg_type", seg_type);
    getInput("expected", expected);

    // String karşılaştırma – switch/case gibi kullanılır
    return (seg_type == expected) ? BT::NodeStatus::SUCCESS
        : BT::NodeStatus::FAILURE;
}

```

Gerçek Örnek: IsTrafficControlActive

```

BT::NodeStatus IsTrafficControlActive::tick()
{

```



```
int tour = 1;
getInput("tour", tour);

// Tur >= 2 ise trafik kontrolleri aktif (şartname kuralı)
return (tour >= 2) ? BT::NodeStatus::SUCCESS
                  : BT::NodeStatus::FAILURE;
}
```

SafetyReflexes'te Condition Kullanım Deseni

```
<!-- Desen: "tehlike varsa dur, yoksa geç" -->
<Fallback>
  <!-- Inverter + Condition = "tehlike YOKSA → SUCCESS → Fallback biter" -->
  <Inverter><PedestrianAhead detected="{detected_pedestrian}" /></Inverter>
  <!-- Buraya ancak yaya VARSA gelinir -->
  <Sequence>
    <StopVehicle />
    <WaitForClear hazard="pedestrian" />
  </Sequence>
</Fallback>
```

Akış:

PedestrianAhead → FAILURE (yaya yok)
→ Inverter → SUCCESS
→ Fallback biter (tehlike yok, devam)

PedestrianAhead → SUCCESS (yaya var!)
→ Inverter → FAILURE
→ Fallback ikinci çocuğa geçer
→ StopVehicle + WaitForClear

9. XML SYNTAX VE TreeNodesModel

9.1 XML Yapısı

```
<?xml version="1.0" encoding="UTF-8"?>
<root main_tree_to_execute="MainTree">

  <!-- Ana ağaç -->
  <BehaviorTree ID="MainTree">
    <Sequence>
      <AksiyonA param="değer" />
      <SubTree ID="AltAgac" _autoremap="true" />
    </Sequence>
  </BehaviorTree>

  <!-- Alt ağaç (SubTree olarak çağrılır) -->
  <BehaviorTree ID="AltAgac">
```

```

    <Fallback>
      <KosulA />
      <AksiyonB />
    </Fallback>
  </BehaviorTree>

  <!-- Port tanımları (Groot uyumluluğu + doğrulama) -->
  <TreeNodesModel>
    <Action ID="AksiyonA">
      <input_port name="param" type="std::string" />
    </Action>
    <Action ID="AksiyonB" />
    <Condition ID="KosulA" />
  </TreeNodesModel>

</root>

```

9.2 TreeNodesModel Detayları

TreeNodesModel, XML'in **sonunda** yer alır ve her custom node'un port'larını bildirir:

```

<TreeNodesModel>
  <!-- Action node: iş yapan -->
  <Action ID="SetMaxSpeed">
    <input_port name="speed" type="double" />
  </Action>

  <!-- Condition node: koşul kontrol eden -->
  <Condition ID="HasMoreSegments">
    <input_port name="seg_index" type="int" />
    <input_port name="route_size" type="int" />
  </Condition>

  <!-- Action with both input and output -->
  <Action ID="LoadMission">
    <input_port name="geojson_file" type="std::string" />
    <input_port name="tour" type="int" />
    <output_port name="route" type="std::string" />
    <output_port name="route_size" type="int" />
    <output_port name="seg_index" type="int" />
  </Action>

  <!-- Bidirectional port -->
  <Action ID="AdvanceSegment">
    <inout_port name="seg_index" type="int" />
  </Action>

  <!-- Port'suz node -->
  <Action ID="StopVehicle" />
</TreeNodesModel>

```

9.3 SubTree ve _autoremap

```
<!-- _autoremap="true": üst ağacın Blackboard'unu alt ağaca otomatik geçir -->
<SubTree ID="SafeDrive" _autoremap="true" />
```

Bu sayede {seg_goal} , {cruise_speed} gibi değişkenler alt ağaçta da erişilebilir.

10. NODE KAYIT (REGISTER) SÜRECİ

10.1 Factory'ye Kayıt

```
// main.cpp veya başlangıç dosyasında:
BT::BehaviorTreeFactory factory;

// Her node'u XML'deki ID ile kaydet
factory.registerNodeType<robotaxi_bt::SetMaxSpeed>("SetMaxSpeed");
factory.registerNodeType<robotaxi_bt::HasMoreSegments>("HasMoreSegments");
// ...

// XML'den ağaç oluştur
auto tree = factory.createTreeFromFile("segment_bt.xml");

// Ağacı çalıştır
BT::NodeStatus result = tree.tickRoot();
```

10.2 Toplu Kayıt Fonksiyonu (Bizim Yaklaşım)

```
// segment_bt_nodes.hpp içinde inline fonksiyon:
inline void registerSegmentBTNodes(BT::BehaviorTreeFactory& factory) {
    // Action node'ları
    factory.registerNodeType<SetMaxSpeed>("SetMaxSpeed");
    factory.registerNodeType<StopVehicle>("StopVehicle");
    factory.registerNodeType<AdvanceSegment>("AdvanceSegment");
    // ...

    // Condition node'ları
    factory.registerNodeType<HasMoreSegments>("HasMoreSegments");
    factory.registerNodeType<IsSegmentType>("IsSegmentType");
    // ...
}
```

10.3 XML ID ↔ C++ Sınıf Eşleşmesi

```
XML'de:      <SetMaxSpeed speed="1.50" />
             ↑
Register:    factory.registerNodeType<SetMaxSpeed>("SetMaxSpeed");
```

```
†
C++:      class SetMaxSpeed : public BT::SyncActionNode { ... };
```

ÖNEMLİ: XML'deki ID, `registerNodeType` 'daki string ve `TreeNodeModel` 'daki ID **birebir aynı** olmalı!

11. YENİ NODE EKLEME — ADIM ADIM KONTROL LİSTESİ

Yeni bir node eklemek için 5 adım:

Adım 1: Tipi Belirle

- Anlık iş mi? → `SyncActionNode`
- Uzun süreli iş mi? → `StatefulActionNode`
- Koşul kontrolü mü? → `ConditionNode`

Adım 2: Header'a Sınıf Ekle (segment_bt_nodes.hpp)

```
class YeniNode : public BT::SyncActionNode {
public:
    YeniNode(const std::string& name, const BT::NodeConfiguration& config)
        : BT::SyncActionNode(name, config) {}

    static BT::PortsList providedPorts() {
        return { BT::InputPort<std::string>("parametre") };
    }

    BT::NodeStatus tick() override;
};
```

Adım 3: Source'a tick() Yaz (segment_bt_nodes.cpp)

```
BT::NodeStatus YeniNode::tick()
{
    std::string parametre;
    getInput("parametre", parametre);
    // İş yap...
    return BT::NodeStatus::SUCCESS;
}
```

Adım 4: Register Fonksiyonuna Ekle (segment_bt_nodes.hpp)

```
factory.registerNodeType<YeniNode>("YeniNode");
```

Adım 5: XML'e Ekle

```
<!-- Ağaçta kullan -->
<YeniNode parametre="{blackboard_degiskeni}" />
```

```
<!-- TreeNodesModel'e port bildir -->
<Action ID="YeniNode">
  <input_port name="parametre" type="std::string" />
</Action>
```

12. SIK YAPILAN HATALAR

Hata 1: SyncActionNode'da RUNNING Dönmek

```
// YANLIŞ! SyncActionNode RUNNING dönmez!
BT::NodeStatus tick() override {
  return BT::NodeStatus::RUNNING; // ← ÇÖKER!
}
// DOĞRU: StatefulActionNode kullan
```

Hata 2: Condition'da Yan Etki

```
// YANLIŞ! Condition state değiştirmemeli!
BT::NodeStatus MyCondition::tick() {
  publisher_>publish(msg); // ← YANLIŞ, bu Action'ın işi
  return BT::NodeStatus::SUCCESS;
}
```

Hata 3: Port Adı Uyuşmazlığı

```
<!-- XML'de "speed" -->
<SetMaxSpeed speed="1.50" />
```

```
// C++'da "hiz" – EŞLEŞMEZ!
BT::InputPort<double>("hiz") // ← XML'deki "speed" ile eşleşmeli
```

Hata 4: getInput Hata Kontrolü Yapmamak

```
// Güvenli kullanım:
auto result = getInput("speed", speed);
if (!result) {
  RCLCPP_ERROR(logger(), "Port okunamadı: %s", result.error().c_str());
  return BT::NodeStatus::FAILURE;
}
```

Hata 5: onHalted'da Temizlik Yapmamak

```
// StatefulActionNode'da onHalted() boş bırakılmamalı
void MyAction::onHalted() {
    // Action client cancel et, timer durdur, GPIO sıfırla vs.
    cancel_action_client();
}
```

ÖZET TABLO: Hangi Node Tipini Kullanmalıyım?

Senaryo	Node Tipi	Örnek
Anlık parametre ayarla	SyncActionNode	SetMaxSpeed, TurnHeadlights
Hemen dur	SyncActionNode	StopVehicle
Loglama	SyncActionNode	LogUnknownSign, RecordParkEntry
Index artır	SyncActionNode	AdvanceSegment
Bayrak temizle	SyncActionNode	ClearHandledFlags
Hedefe sür (sürekli)	StatefulActionNode	FollowLaneSegment
Süre bekle	StatefulActionNode	Dwell, WaitForClear
Sinyal bekle	StatefulActionNode	WaitForGoSignal, WaitForGreenLight
Park manevrası	StatefulActionNode	ExecuteParking
Değer karşılaştır	ConditionNode	IsSegmentType, IsLightRed
Sayı kontrolü	ConditionNode	HasMoreSegments
Tur kontrolü	ConditionNode	IsTrafficControlActive
Sensör durumu	ConditionNode	PedestrianAhead, EmergencyStop

EK: Projedeki Tüm Node'ların Durumu

✅ Tam İmplemente (Gerçek mantık çalışır)

- HasMoreSegments, IsSegmentType, IsTrafficControlActive
- IsLightRed, IsRoadSignType
- AdvanceSegment, Dwell, LogUnknownSign

✂ Basit Stub (ROS bağlantısı eklenecek)

- SetMaxSpeed, StopVehicle, TurnHeadlights
- ClearHandledFlags, SignalPassengerEvent
- RecordMissionPoint, RecordParkEntryReached

🔧 Sensör/Algılama Bağımlı (Takım dolduracak)

- PedestrianAhead, DynamicObstacleAhead, StaticObstacleInLane
- TrafficLightAhead, StopSignAhead, GlobalRoadSignAhead

- EmergencyStopRequested, IsStuck

Segment Haritası Bağımlı (Harita gelince)

- LoadMission, GetCurrentSegment, ReplanRoute
- FindParkingSlot, CalculateLaneChange